

使用 MCP Toolbox for Databases 和 Agent Development Kit (ADK) 建構旅遊代理

來源：<https://codelabs.developers.google.com/travel-agent-mcp-toolbox-adk?hl=zh-tw>

步驟數：11

本程式碼研究室將引導您建立旅遊服務專員，以便回答使用者有關城市內飯店的查詢。代理程式會使用代理開發套件 (ADK) 建構，並利用資料庫的 MCP Toolbox，讓代理程式存取資料庫中的資料。

目錄

1. [簡介](#)
2. [事前準備](#)
3. [建立 Cloud SQL 執行個體](#)
4. [準備 Hotels 資料庫](#)
5. [設定 MCP Toolbox for Databases](#)
6. [在 Gemini CLI 中整合 MCP Toolbox](#)
7. [使用 Agent Development Kit \(ADK\) 編寫 AI 代理](#)
8. [將代理程式連結至工具](#)
9. [\(選用\) 將 MCP Toolbox for Databases 和 Agent 部署至 Cloud Run](#)
10. [清除所用資源](#)
11. [恭喜](#)

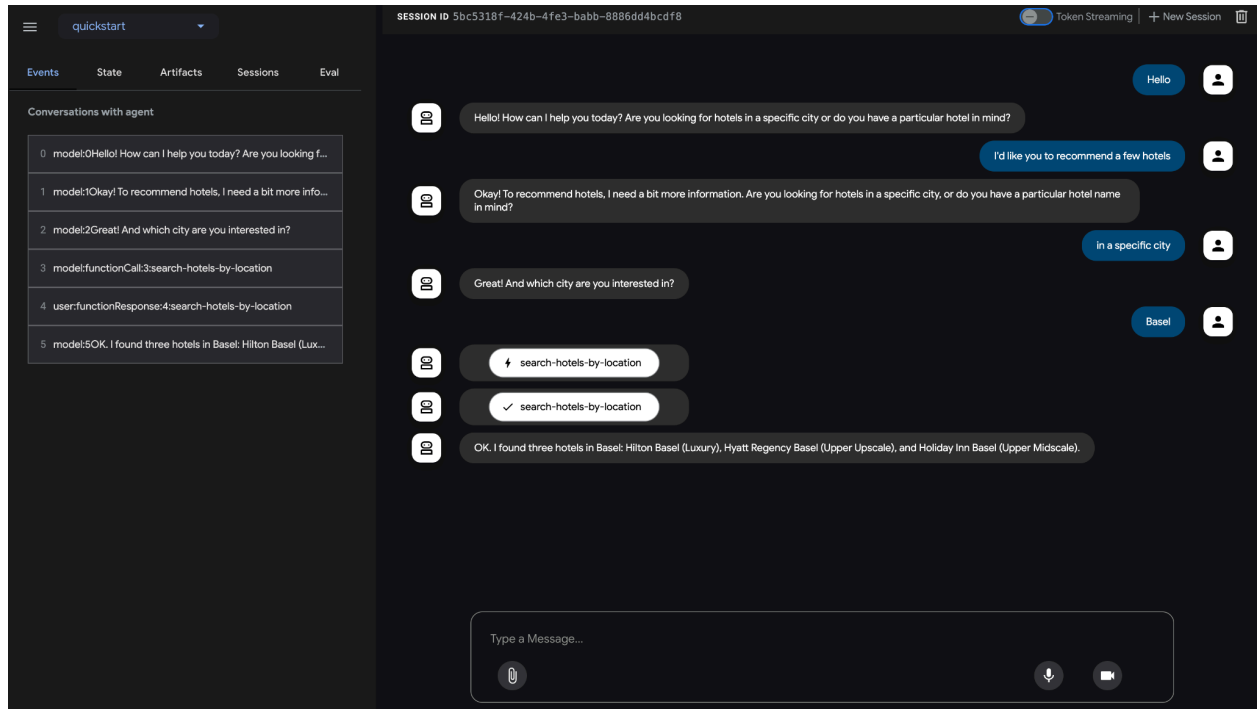
步驟 1 · 約 5 分鐘

1. 簡介

在本程式碼實驗室中，您將使用 [Agent Development Kit \(ADK\)](#) 建構代理，並運用 [MCP Toolbox for Databases](#)。

在本程式碼研究室中，您將逐步完成下列步驟：

1. 佈建 PostgreSQL 適用的 Cloud SQL 資料庫，其中包含飯店資料庫和範例資料。
2. 設定 MCP Toolbox for Databases，以便存取資料。
3. 使用 Agent Development Kit (ADK) 設計及開發代理，該代理會運用 MCP Toolbox 回答使用者的查詢。
4. 探索透過 Cloud Run 服務在本機和 Google Cloud 測試代理程式和 MCP Toolbox for Databases 的選項。



執行步驟

- 設計、建構及部署代理程式，回答使用者對某個地點飯店的查詢，或依名稱搜尋飯店。

課程內容

- 佈建 PostgreSQL 適用的 Cloud SQL 資料庫，並填入範例資料。
- 為 PostgreSQL 適用的 Cloud SQL 資料庫執行個體設定 MCP Toolbox for Databases。
- 使用 Agent Development Kit (ADK) 設計及開發代理，回答使用者查詢。
- 在本機環境中測試 Agent 和 MCP Toolbox for Databases。
- (選用) 在 Google Cloud 中部署 Agent 和 MCP Toolbox for Databases。

軟硬體需求

- Chrome 網路瀏覽器
- Gmail 帳戶
- 已啟用計費功能的 Cloud 專案

本程式碼研究室適合各種程度的開發人員 (包括初學者)，並使用 Python 撰寫範例應用程式。不過，您不需要具備 Python 知識，只要有基本的程式碼閱讀能力，就足以瞭解課程中介紹的概念。

2. 事前準備

Google Cloud 抵免額：如要開始使用，請透過這個[連結](#)兌換免費的 Google Cloud 抵免額。請按照[這裡](#)的操作說明啟用抵免額並建立新專案。

建立專案

1. 在 [Google Cloud 控制台](#) 的專案選取器頁面中，選取或建立 Google Cloud [專案](#)。
2. 確認 Cloud 專案已啟用計費功能。瞭解如何[檢查專案是否已啟用計費功能](#)。
3. 您將使用 [Cloud Shell](#)，這是 Google Cloud 中執行的指令列環境，預先載入了 bq。按一下 Google Cloud 控制台頂端的「啟用 Cloud Shell」。



4. 連至 Cloud Shell 後，請使用下列指令確認驗證已完成，專案也已設為獲派的專案 ID：

〇〇〇〇

```
gcloud auth list
```

5. 在 Cloud Shell 中執行下列指令，確認 gcloud 指令已瞭解您的專案。

```
gcloud config list project
```

6. 如果未設定專案，請使用下列指令來設定：

```
gcloud config set project <YOUR_PROJECT_ID>
```

7. 透過下列指令啟用必要的 API。這可能需要幾分鐘的時間，請耐心等待。

```
gcloud services enable cloudresourcemanager.googleapis.com \
    servicenetworking.googleapis.com \
    run.googleapis.com \
    cloudbuild.googleapis.com \
    cloudfunctions.googleapis.com \
    aiplatform.googleapis.com \
    sqladmin.googleapis.com \
    compute.googleapis.com
```

成功執行指令後，您應該會看到類似下方的訊息：

```
Operation "operations/..." finished successfully.
```

除了使用 gcloud 指令，您也可以透過控制台搜尋各項產品，或使用這個[連結](#)。

如果遺漏任何 API，您隨時可以在導入過程中啟用。

如要瞭解 gcloud 指令和用法，請參閱[說明文件](#)。

步驟 3 · 約 10 分鐘

3. 建立 Cloud SQL 執行個體

我們將使用 [PostgreSQL 適用的 Cloud SQL](#) 執行個體儲存飯店資料。PostgreSQL 適用的 Cloud SQL 是一項全代管資料庫服務，可協助您在 Google Cloud Platform 中設定、維護及管理 PostgreSQL 關聯資料庫。

在 Cloud Shell 中執行下列指令，即可建立執行個體：

```
gcloud sql instances create hoteldb-instance \  
--database-version=POSTGRES_15 \  
--tier db-g1-small \  
--region=us-central1 \  
--edition=ENTERPRISE \  
--root-password=postgres
```

這項指令大約需要 3 到 5 分鐘才能執行完畢。指令成功執行後，您應該會看到輸出內容，指出指令已完成，以及 Cloud SQL 執行個體資訊，例如 NAME、DATABASE_VERSION、LOCATION 等。

步驟 4 · 約 5 分鐘

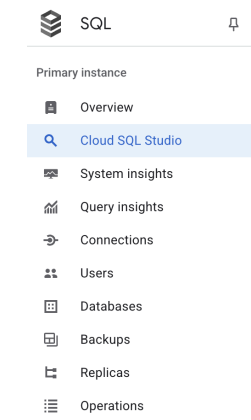
4. 準備 Hotels 資料庫

現在，我們要為飯店代理建立一些範例資料。

前往 Cloud 控制台的 [Cloud SQL 頁面](#)，您應該會看到 `hotelldb-instance` 已準備就緒並建立完成。按一下執行個體的名稱 (`hotelldb-instance`)，如下所示：

☐	☒	hotelldb-instance	Enterprise
--------------------------	-------------------------------------	-------------------	------------

在 Cloud SQL 左側選單中，前往 `Cloud SQL Studio` 選單選項，如下所示：



系統會要求您登入 Cloud SQL Studio，我們將透過這個工具提供幾個 SQL 指令。選取「Database」選項的 `postgres`，並將「User」和「Password」的值設為 `postgres`。按一下 `AUTHENTICATE`。

首先，請按照下方結構定義建立飯店資料表。在 Cloud SQL Studio 的其中一個「Editor」（編輯器）窗格中，執行下列 SQL：

```
CREATE TABLE hotels(  
  id          INTEGER NOT NULL PRIMARY KEY,  
  name        VARCHAR NOT NULL,  
  location    VARCHAR NOT NULL,  
  price_tier  VARCHAR NOT NULL,  
  checkin_date DATE    NOT NULL,  
  checkout_date DATE    NOT NULL,  
  booked      BIT      NOT NULL  
);
```

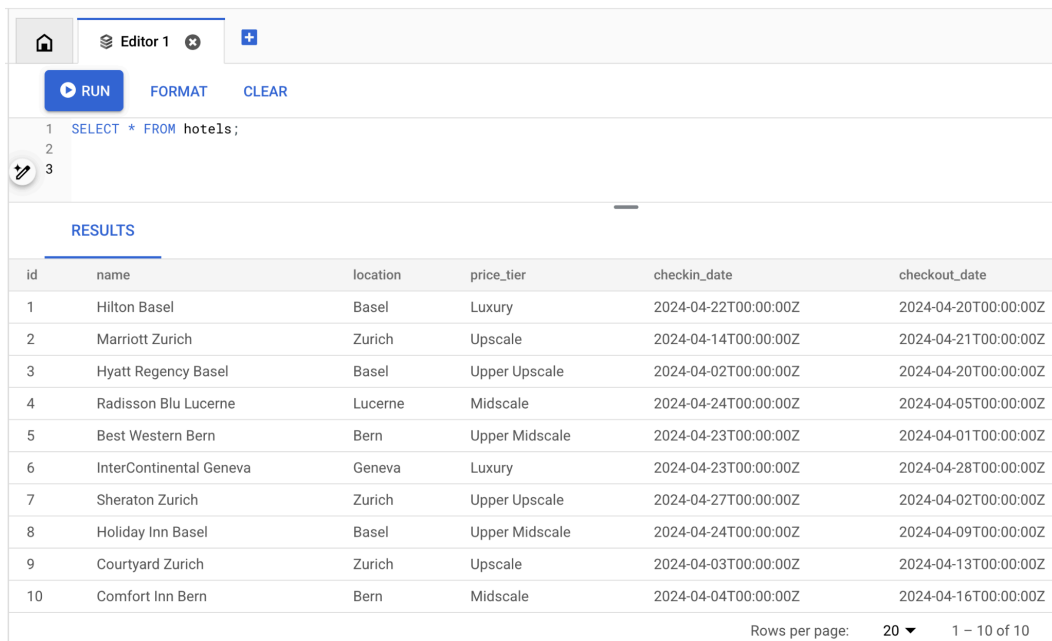
現在，我們來在飯店資料表中填入範例資料。執行下列 SQL：


```
INSERT INTO hotels(id, name, location, price_tier, checkin_date, checkout_date, booked)
VALUES
(1, 'Hilton Basel', 'Basel', 'Luxury', '2024-04-20', '2024-04-22', B'0'),
(2, 'Marriott Zurich', 'Zurich', 'Upscale', '2024-04-14', '2024-04-21', B'0'),
(3, 'Hyatt Regency Basel', 'Basel', 'Upper Upscale', '2024-04-02', '2024-04-20', B'0'),
(4, 'Radisson Blu Lucerne', 'Lucerne', 'Midscale', '2024-04-05', '2024-04-24', B'0'),
(5, 'Best Western Bern', 'Bern', 'Upper Midscale', '2024-04-01', '2024-04-23', B'0'),
(6, 'InterContinental Geneva', 'Geneva', 'Luxury', '2024-04-23', '2024-04-28', B'0'),
(7, 'Sheraton Zurich', 'Zurich', 'Upper Upscale', '2024-04-02', '2024-04-27', B'0'),
(8, 'Holiday Inn Basel', 'Basel', 'Upper Midscale', '2024-04-09', '2024-04-24', B'0'),
(9, 'Courtyard Zurich', 'Zurich', 'Upscale', '2024-04-03', '2024-04-13', B'0'),
(10, 'Comfort Inn Bern', 'Bern', 'Midscale', '2024-04-04', '2024-04-16', B'0');
```

請執行 SELECT SQL 驗證資料，如下所示：

```
SELECT * FROM hotels;
```

您應該會看到飯店資料表中的多筆記錄，如下所示：



The screenshot shows a SQL editor interface. At the top, there's a toolbar with a home icon, a tab labeled 'Editor 1', and a plus icon. Below the toolbar are buttons for 'RUN', 'FORMAT', and 'CLEAR'. The SQL editor area contains the query: `1 SELECT * FROM hotels;`. Below the editor, there's a 'RESULTS' section. It contains a table with 6 columns: 'id', 'name', 'location', 'price_tier', 'checkin_date', and 'checkout_date'. The table has 10 rows of data. At the bottom right of the results section, it says 'Rows per page: 20' and '1 - 10 of 10'.

id	name	location	price_tier	checkin_date	checkout_date
1	Hilton Basel	Basel	Luxury	2024-04-22T00:00:00Z	2024-04-20T00:00:00Z
2	Marriott Zurich	Zurich	Upscale	2024-04-14T00:00:00Z	2024-04-21T00:00:00Z
3	Hyatt Regency Basel	Basel	Upper Upscale	2024-04-02T00:00:00Z	2024-04-20T00:00:00Z
4	Radisson Blu Lucerne	Lucerne	Midscale	2024-04-24T00:00:00Z	2024-04-05T00:00:00Z
5	Best Western Bern	Bern	Upper Midscale	2024-04-23T00:00:00Z	2024-04-01T00:00:00Z
6	InterContinental Geneva	Geneva	Luxury	2024-04-23T00:00:00Z	2024-04-28T00:00:00Z
7	Sheraton Zurich	Zurich	Upper Upscale	2024-04-27T00:00:00Z	2024-04-02T00:00:00Z
8	Holiday Inn Basel	Basel	Upper Midscale	2024-04-24T00:00:00Z	2024-04-09T00:00:00Z
9	Courtyard Zurich	Zurich	Upscale	2024-04-03T00:00:00Z	2024-04-13T00:00:00Z
10	Comfort Inn Bern	Bern	Midscale	2024-04-04T00:00:00Z	2024-04-16T00:00:00Z

我們已完成 Cloud SQL 執行個體設定程序，並建立範例資料。在下一節中，我們將設定 MCP Toolbox for Databases。

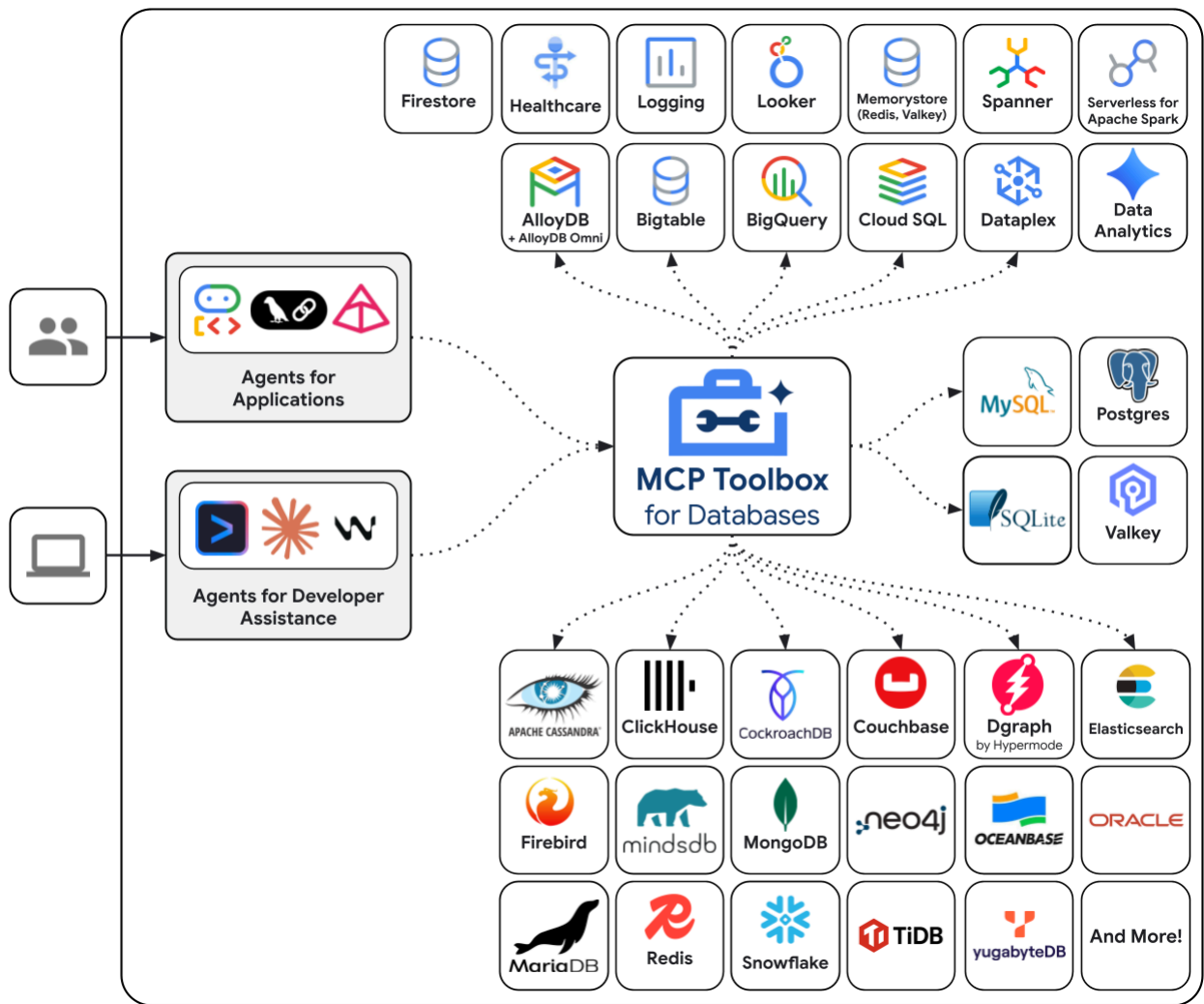
5. 設定 MCP Toolbox for Databases

MCP Toolbox for Databases 是資料庫專用的開放原始碼 MCP 伺服器，專為企業級和生產品質而設計。這項服務可處理連線集區和驗證等複雜作業，讓您更輕鬆、快速且安全地開發工具。

您可以透過 Toolbox 建構生成式 AI 工具，讓代理程式存取資料庫中的資料。工具箱提供以下功能：

- 簡化開發作業：在不到 10 行的程式碼中將工具整合至代理程式、在多個代理程式或架構之間重複使用工具，以及更輕鬆地部署新版工具。
- 提升效能：連線集區、驗證等最佳做法。
- 強化安全性：整合式驗證機制，可更安全地存取資料
- 端對端可觀測性：內建支援 OpenTelemetry，可直接使用指標和追蹤功能。

Toolbox 位於應用程式的協調架構和資料庫之間，提供用於修改、發布或叫用工具的控制層。您可以在集中位置儲存及更新工具，並在代理程式和應用程式之間共用工具，不必重新部署應用程式就能更新工具，簡化工具管理作業。



您可以看到，MCP Toolbox for Databases 支援的資料庫之一是 Cloud SQL，我們已在上一節中佈建該資料庫。

安裝 Toolbox

開啟 Cloud Shell 終端機，然後建立名為 `mcp-toolbox` 的資料夾。

```
mkdir mcp-toolbox
```

透過下列指令前往 **mcp-toolbox** 資料夾：

```
cd mcp-toolbox
```

透過下列指令碼安裝 MCP Toolbox for Databases 的二進位版本。下列指令適用於 Linux，但如果您使用 Mac 或 Windows，請務必下載正確的二進位檔。請查看[作業系統和架構的發布頁面](#)，並下載正確的二進位檔。

```
export VERSION=1.1.0
curl -L -o toolbox https://storage.googleapis.com/mcp-toolbox-for-
databases/v$VERSION/linux/amd64/toolbox
chmod +x toolbox
```

現在我們已準備好工具箱的二進位版本，請驗證我們是否已正確設定工具箱二進位檔，且該二進位檔是否指出正確版本。

輸入下列指令，判斷 Toolbox 的版本：

```
./toolbox -v
```

這會輸出類似以下的內容：

```
toolbox version 1.1.0+binary.linux.amd64.466aef0
```

下一步是使用資料來源和其他設定來設定工具箱。

設定 tools.yaml

設定 Toolbox 的主要方式是透過 **tools.yaml** 檔案。在同一個資料夾 (即 **mcp-toolbox**) 中建立名為 **tools.yaml** 的檔案，內容如下所示。

您可以使用 Cloud Shell 提供的 nano 編輯器。nano 指令如下：「**nano tools.yaml**」。

請記得將 **YOUR_PROJECT_ID** 值替換為您的 Google Cloud 專案 ID。

```

kind: source
name: my-cloud-sql-source
type: cloud-sql-postgres
project: YOUR_PROJECT_ID
region: us-central1
instance: hoteldb-instance
database: postgres
user: postgres
password: postgres
---
kind: tool
name: search-hotels-by-name
type: postgres-sql
source: my-cloud-sql-source
description: Search for hotels based on name.
parameters:
  - name: name
    type: string
    description: The name of the hotel.
statement: SELECT * FROM hotels WHERE name ILIKE '%' || $1 || '%';
---
kind: tool
name: search-hotels-by-location
type: postgres-sql
source: my-cloud-sql-source
description: Search for hotels based on location. Result is sorted by price from least to
most expensive.
parameters:
  - name: location
    type: string
    description: The location of the hotel.
statement: |
  SELECT *
  FROM hotels
  WHERE location ILIKE '%' || $1 || '%'
  ORDER BY
    CASE price_tier
      WHEN 'Midscale' THEN 1
      WHEN 'Upper Midscale' THEN 2
      WHEN 'Upscale' THEN 3
      WHEN 'Upper Upscale' THEN 4
      WHEN 'Luxury' THEN 5
      ELSE 99 -- Handle any unexpected values, place them at the end
    END;
---
kind: toolset
name: my_first_toolset
tools:
  - search-hotels-by-name
  - search-hotels-by-location

```

簡要瞭解檔案：

1. **Sources** 代表工具可互動的不同資料來源。來源代表工具可互動的資料來源。您可以在 `tools.yaml` 檔案的來源區段中，將 **Sources** 定義為對應。一般來說，來源設定會包含連線至資料庫及與資料庫互動所需的任何資訊。在本例中，我們已設定單一來源，並使用憑證指向 PostgreSQL 適用的 Cloud SQL 執行個體。詳情請參閱「[來源](#)」參考資料。
2. **Tools** 定義代理程式可執行的動作，例如讀取及寫入來源。工具代表代理程式可執行的動作，例如執行 SQL 陳述式。您可以在 `tools.yaml` 檔案的工具區段中，將 **Tools** 定義為對應。一般來說，工具需要來源才能執行動作。在本例中，我們定義了 `search-hotels-by-name` 和 `search-hotels-by-location` 這兩項工具，並指定工具執行的來源，以及 SQL 和參數。詳情請參閱「[工具](#)」參考資料。
3. 最後是 **Toolset**，可讓您定義要一起載入的工具群組。這項功能有助於根據代理程式或應用程式定義不同群組。在本例中，我們有一個名為 `my_first_toolset` 的單一工具集，其中包含我們定義的兩項工具。

在 nano 編輯器中，按照下列步驟儲存 `tools.yaml` 檔案：

- 按下 `Ctrl + O` (「寫出」指令)。
- 系統會要求您確認「要寫入的檔案名稱」。按下 `Enter` 鍵即可開啟。
- 按下 `Ctrl + X` 即可退出。

執行 MCP Toolbox for Databases 伺服器

從 `mcp-toolbox` 資料夾執行下列指令，啟動伺服器：

```
./toolbox --config "tools.yaml"
```

理想情況下，您應該會看到伺服器已連線至資料來源，並已載入工具集和工具的輸出內容。輸出範例如下所示：

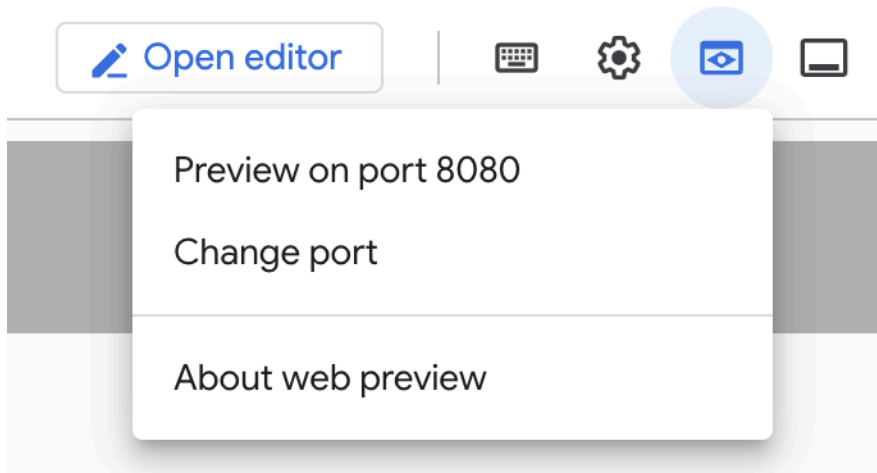
```
2026-04-25T09:08:00.738271-07:00 INFO "Initialized 1 sources: my-cloud-sql-source"
2026-04-25T09:08:00.738397-07:00 INFO "Initialized 0 authServices: "
2026-04-25T09:08:00.738405-07:00 INFO "Initialized 0 embeddingModels: "
2026-04-25T09:08:00.738453-07:00 INFO "Initialized 2 tools: search-hotels-by-name, search-hotels-by-location"
2026-04-25T09:08:00.738497-07:00 INFO "Initialized 2 toolsets: my_first_toolset, default"
2026-04-25T09:08:00.738504-07:00 INFO "Initialized 0 prompts: "
2026-04-25T09:08:00.738517-07:00 INFO "Initialized 1 promptsets: default"
2026-04-25T09:08:00.738566-07:00 WARN "wildcard (`*`) allows all origin to access the resource and is not secure. Use it with cautious for public, non-sensitive data, or during local development. Recommended to use `--allowed-origins` flag"
2026-04-25T09:08:00.738625-07:00 WARN "wildcard (`*`) allows all hosts to access the resource and is not secure. Use it with cautious for public, non-sensitive data, or during local development. Recommended to use `--allowed-hosts` flag to prevent DNS rebinding attacks"
2026-04-25T09:08:00.738909-07:00 INFO "Server ready to serve!"
```

MCP Toolbox 伺服器預設會在通訊埠 5000 上執行。如果發現通訊埠 5000 已在使用中，請隨意使用其他通訊埠 (例如 7000)，如下列指令所示。請在後續指令中使用 7000，而非 5000 連接埠。

```
./toolbox --config "tools.yaml" --port 7000
```

我們將使用 Cloud Shell 測試這項功能。

在 Cloud Shell 中點選「網頁預覽」，如下所示：



按一下「變更通訊埠」，將通訊埠設為 5000 (如下所示)，然後按一下「變更並預覽」。

A screenshot of a 'Change Preview Port' dialog box. The dialog has a title bar that says 'Change Preview Port'. Inside, there is a text input field labeled 'Port number*' with the value '5000' entered. Below the input field, there are two buttons: 'Cancel' and 'Change and Preview'.

這應該會產生下列輸出內容：

 Hello, World! 

透過 MCP Toolbox for Databases 使用者介面測試工具

工具箱提供視覺化介面 (**工具箱 UI**)，可直接與工具互動，方法是在簡單的網頁 UI 中修改參數、管理標頭及執行呼叫。
如要測試，請執行先前用來啟動 Toolbox 伺服器的指令，並加上 `--ui` 選項。
如要這麼做，請關閉先前可能正在執行的 MCP Toolbox for Databases 伺服器執行個體，然後輸入下列指令：

```
./toolbox --config "tools.yaml" --ui
```

理想情況下，您應該會看到伺服器已連線至資料來源，並已載入工具集和工具的輸出內容。以下是輸出範例，您會發現其中提到 Toolbox UI 正在運作。

```
2026-04-25T09:17:45.54415-07:00 INFO "Initialized 1 sources: my-cloud-sql-source"
2026-04-25T09:17:45.544225-07:00 INFO "Initialized 0 authServices: "
2026-04-25T09:17:45.544239-07:00 INFO "Initialized 0 embeddingModels: "
2026-04-25T09:17:45.544298-07:00 INFO "Initialized 2 tools: search-hotels-by-location,
search-hotels-by-name"
2026-04-25T09:17:45.544345-07:00 INFO "Initialized 2 toolsets: default, my_first_toolset"
2026-04-25T09:17:45.544365-07:00 INFO "Initialized 0 prompts: "
2026-04-25T09:17:45.544393-07:00 INFO "Initialized 1 promptsets: default"
2026-04-25T09:17:45.544439-07:00 WARN "wildcard (`*`) allows all origin to access the
resource and is not secure. Use it with cautious for public, non-sensitive data, or during
local development. Recommended to use `--allowed-origins` flag"
2026-04-25T09:17:45.54448-07:00 WARN "wildcard (`*`) allows all hosts to access the resource
and is not secure. Use it with cautious for public, non-sensitive data, or during local
development. Recommended to use `--allowed-hosts` flag to prevent DNS rebinding attacks"
2026-04-25T09:17:45.544887-07:00 INFO "Server ready to serve!"
2026-04-25T09:17:45.544908-07:00 INFO "Toolbox UI is up and running at:
http://127.0.0.1:5000/ui"
```

按一下使用者介面網址，**確認您擁有**

`/ui`

網址結尾 (如果您在 Cloud Shell 中執行這項操作，瀏覽器重新導向會導致結尾沒有 /ui)。系統會顯示如下所示的 UI：



Tools

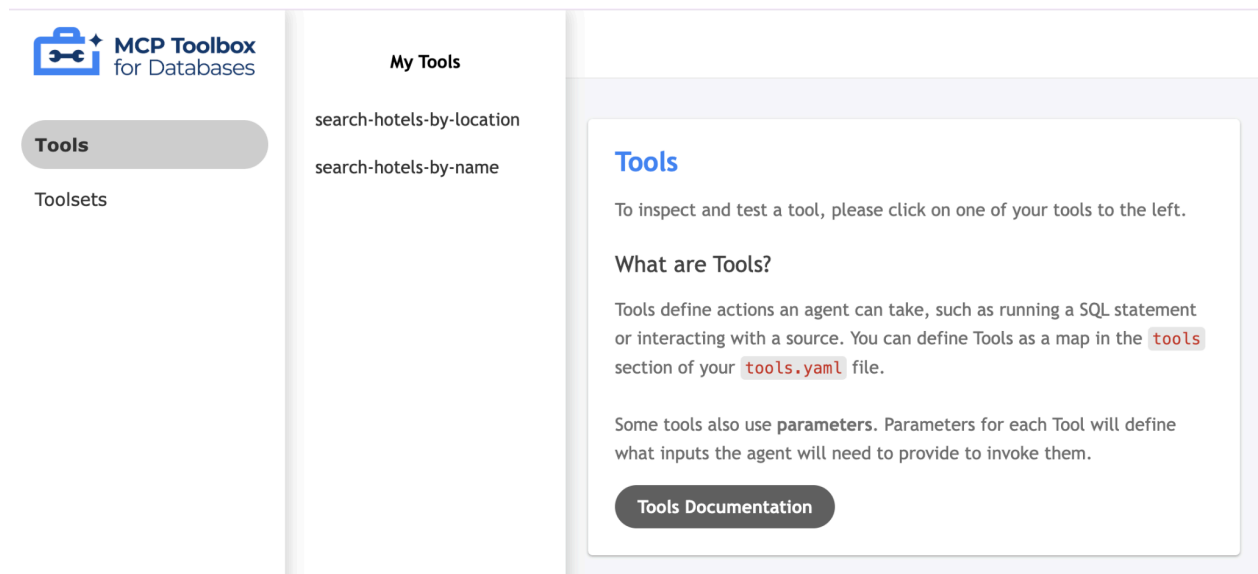
Toolsets

Welcome to Toolbox UI

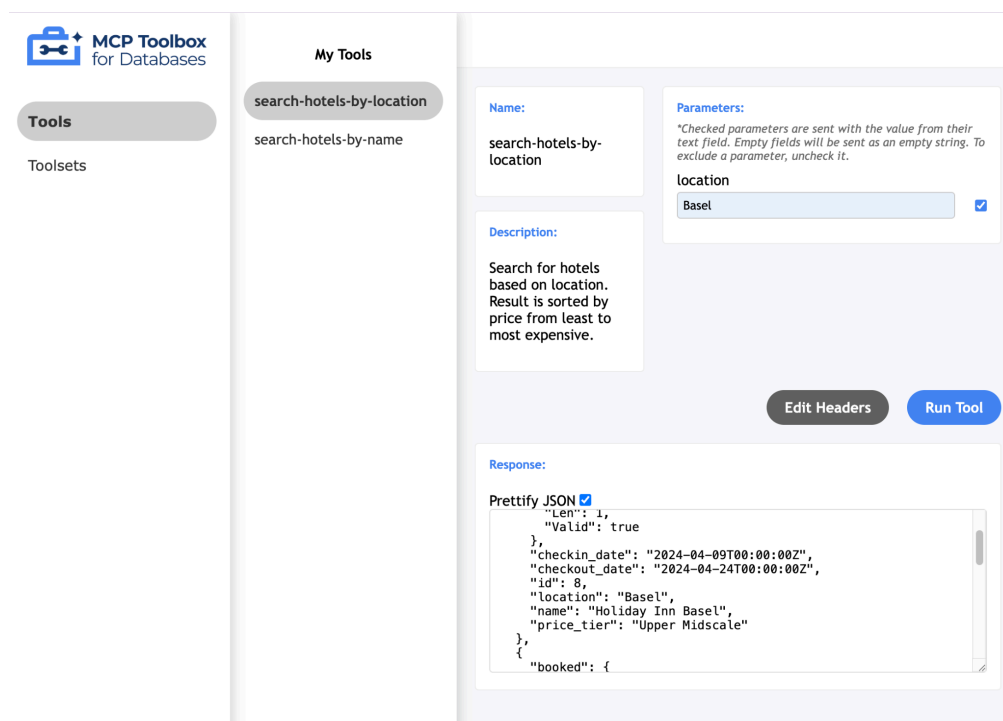
Toolbox UI is a built-in web interface that allows users to visually inspect and test out configured resources such as tools and toolsets. To get started, select a resource from the navigation tab to the left.

[Toolbox UI Documentation](#)

按一下左側的「工具」選項，即可查看已設定的工具。在本例中，應該會有兩個工具，即 `search-hotels-by-name` 和 `search-hotels-by-location`，如下所示：

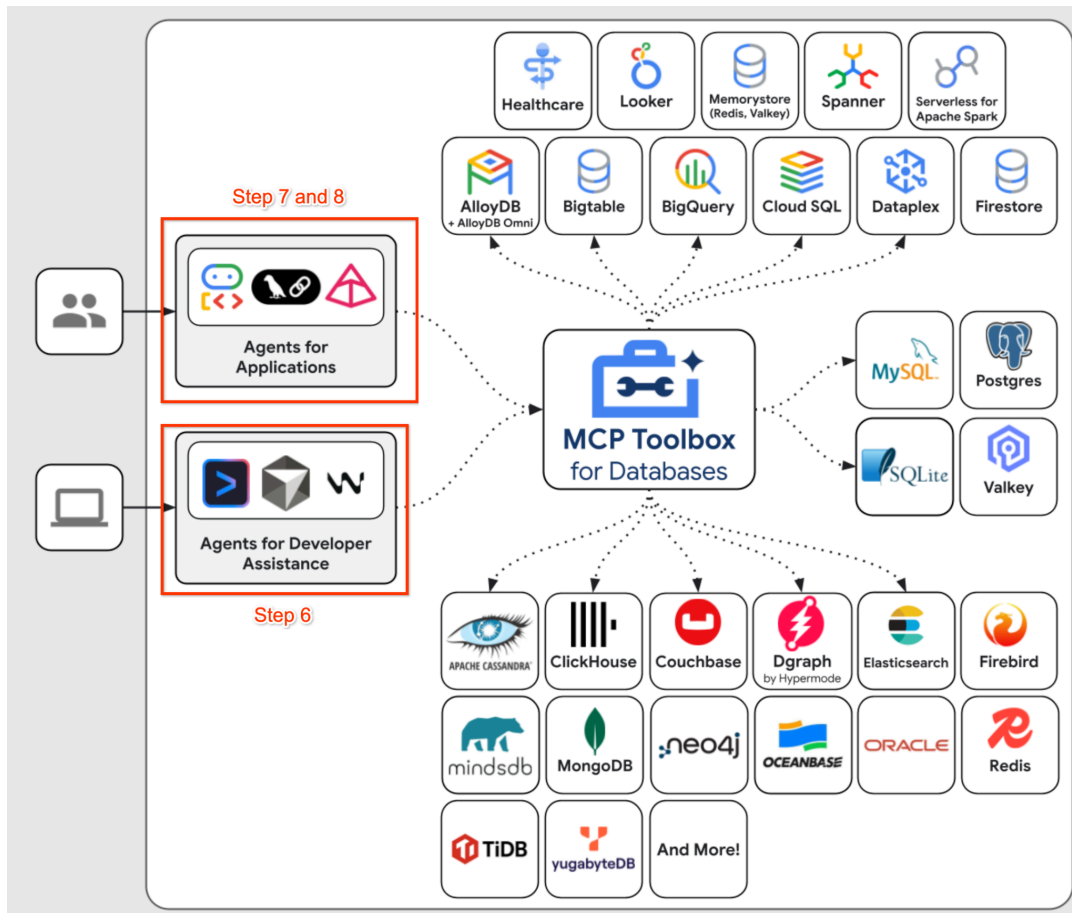


只要點選其中一個工具 (**search-hotels-by-location**)，系統就會顯示頁面，讓您提供必要參數值來測試工具，然後點選「執行工具」即可查看結果。以下是執行範例：



MCP Toolkit for Databases 也說明瞭以 Python 驗證及測試工具的方法，詳情請參閱[這篇文章](#)。

回顧稍早的圖表 (如下所示)，我們已完成資料庫和 MCP 伺服器的設定，現在有兩條路可走：



1. 如要瞭解如何將 MCP 伺服器設定為 AI 輔助終端機 / IDE，請參閱步驟 6。本節將說明如何將 MCP 工具箱伺服器整合至 **Gemini CLI**。
2. 如要瞭解如何使用 **Agent Development Kit (ADK) using Python**、編寫可將 MCP 伺服器工具箱當做工具的代理，以及回答與資料集相關的問題，請前往步驟 7 和 8。

6. 在 Gemini CLI 中整合 MCP Toolbox

Gemini CLI 是開放原始碼 AI 代理，可讓您直接在終端機中使用 Gemini 的強大功能。這項工具可用於編碼和非編碼工作。這項工具已整合多種工具，並支援 MCP 伺服器。

我們已建立可正常運作的 MCP 伺服器，因此本節的目標是在 Gemini CLI 中設定 MCP Toolbox for Databases 伺服器，然後使用 Gemini CLI 與資料互動。

首先，請確認您是否已在其中一個 Cloud Shell 終端機中啟動並執行 Toolbox。假設您在預設通訊埠 **5000** 上執行，MCP 伺服器介面會位於下列端點：**`http://localhost:5000/mcp`**。

開啟新終端機，然後依下列方式建立名為 **`my-gemini-cli-project`** 的資料夾。前往 **`my-gemini-cli-project`** 資料夾。

```
mkdir my-gemini-cli-project
cd my-gemini-cli-project
```

執行下列指令，將 MCP 伺服器新增至 Gemini CLI 中設定的 MCP 伺服器清單。

```
gemini mcp add --scope="project" --transport="http" "MCPToolbox" "http://localhost:5000/mcp"
```

你可以透過下列指令，查看 Gemini CLI 中設定的 MCP 伺服器目前清單：

```
gemini mcp list
```

理想情況下，您應該會看到我們設定的 MCPToolbox 旁邊有綠色勾號，表示 Gemini CLI 已連線至 MCP 伺服器。

```
Configured MCP servers:

✓ MCPToolbox: http://localhost:5000/mcp (http) - Connected
```

如要查看 MCP 伺服器的設定位置，請查看 **`my-gemini-cli-project`** 資料夾中的 **`.gemini`** 資料夾。其中包含已設定 MCP 伺服器的 **`settings.json`** 檔案。

在推出 Gemini CLI 之前，您可能需要設定下列環境變數，協助 Gemini CLI 將要求傳送至正確的模型。

```
export GOOGLE_CLOUD_PROJECT=YOUR_GOOGLE_CLOUD_PROJECT_ID
export GOOGLE_CLOUD_LOCATION=global
```

在同一個終端機中，確認您位於 **`my-gemini-cli-project`** 資料夾。透過 **`gemini`** 指令啟動 Gemini CLI。

這時會顯示 Gemini CLI 介面，您會看到現在已設定 1 個 MCP 伺服器。您可以使用 **`/mcp list`** 指令查看 MCP 伺服器和工具清單。例如，以下是輸出內容範例：

```
> /mcp list
```

iConfigured MCP servers:

 **MCPToolbox** - Ready (2 tools)
Tools:
- search-hotels-by-location
- search-hotels-by-name

現在可以提供下列任一提示：

1. Which hotels are there in Basel?
2. Tell me more about the Hyatt Regency?

你會發現，上述查詢會導致 Gemini CLI 從 MCPToolbox 選取適當的工具。系統會要求您授予執行工具的權限。授予必要權限後，您會發現結果是從資料庫傳回。

步驟 7 · 約 20 分鐘

7. 使用 Agent Development Kit (ADK) 編寫 AI 代理

安裝 Agent Development Kit (ADK)

在 Cloud Shell 中開啟新的終端機分頁，然後建立名為 `my-agents` 的資料夾，如下所示。前往 `my-agents` 資料夾。

```
mkdir my-agents
cd my-agents
```

現在，讓我們使用 `venv` 建立虛擬 Python 環境，如下所示：

```
python -m venv .venv
```

按照下列步驟啟動虛擬環境：

```
source .venv/bin/activate
```

安裝 ADK 和 MCP Toolbox for Databases 套件，以及 langchain 依附元件，如下所示：

```
pip install google-adk toolbox-core
```

現在可以按照下列方式叫用 `adk` 公用程式。

```
adk
```

系統會顯示指令清單。

```
Usage: adk [OPTIONS] COMMAND [ARGS]...
  Agent Development Kit CLI tools.

Options:
  --version  Show the version and exit.
  --help     Show this message and exit.

Commands:
  api_server  Starts a FastAPI server for agents.
  conformance Conformance testing tools for ADK.
  create      Creates a new app in the current folder with prepopulated agent template.
  deploy      Deploys agent to hosted environments.
  eval        Evaluates an agent given the eval sets.
  eval_set    Manage Eval Sets.
  migrate     ADK migration commands.
  optimize    Optimizes the root agent instructions using the GEPA optimizer.
  run         Runs an interactive CLI for a certain agent.
  web         Starts a FastAPI server with Web UI for agents.
```

建立第一個代理程式應用程式

我們現在要使用 `adk`，透過 `adk create` 指令，為飯店代理程式應用程式建立基本架構，並使用下列應用程式名稱 `** (hotel_agent_app) **`。

```
adk create hotel_agent_app
```

按照步驟選取下列項目：

- Gemini 模型，用於為根代理程式選擇模型。
- 選擇 Vertex AI 做為後端。
- 畫面上會顯示預設的 Google 專案 ID 和區域。選取預設值本身。

```
Choose a model for the root agent:
```

1. gemini-2.5-flash
2. Other models (fill later)

```
Choose model (1, 2): 1
```

1. Google AI
2. Vertex AI

```
Choose a backend (1, 2): 2
```

```
You need an existing Google Cloud account and project, check out this link for details:
https://google.github.io/adk-docs/get-started/quickstart/#gemini---google-cloud-vertex-ai
```

```
Enter Google Cloud project ID [YOUR_PROJECT_ID]:
```

```
Enter Google Cloud region [us-central1]:
```

```
Agent created in <YOUR_HOME_FOLDER>/my-agents/hotel_agent_app:
```

- .env
- __init__.py
- agent.py

觀察系統為 Agent 建立預設範本和必要檔案的資料夾。您可以在 `<YOUR_HOME_FOLDER>/my-agents/hotel_agent_app` 目錄的終端機中，透過 `ls -al` 指令查看檔案。

首先是 `.env` 檔案。如先前所述，這個檔案已建立完成，您只要查看下方顯示的檔案內容即可：

```
GOOGLE_GENAI_USE_VERTEXAI=1
GOOGLE_CLOUD_PROJECT=YOUR_GOOGLE_PROJECT_ID
GOOGLE_CLOUD_LOCATION=YOUR_GOOGLE_PROJECT_REGION
```

這些值表示我們將透過 Vertex AI 使用 Gemini，以及 Google Cloud 專案 ID 和位置的相應值。

接著是 `__init__.py` 檔案，這個檔案會將資料夾標示為模組，並包含從 `agent.py` 檔案匯入代理程式的單一陳述式。

```
from . import agent
```

最後，我們來看看 `agent.py` 檔案。內容如下所示：

```
from google.adk.agents import Agent

root_agent = Agent(
    model='gemini-2.5-flash',
    name='root_agent',
    description='A helpful assistant for user questions.',
    instruction='Answer user questions to the best of your knowledge',
)
```

這是您可以使用 ADK 撰寫的最簡單代理程式。根據 ADK 說明文件[頁面](#)，代理程式是獨立的執行單元，可自主運作以達成特定目標。代理程式可以執行工作、與使用者互動、使用外部工具，以及與其他代理程式協調。

具體來說，LLM Agent (通常別名為 Agent) 會以大型語言模型 (LLM) 做為核心引擎，理解自然語言、推論、規劃、生成回覆，並動態決定後續步驟或要使用的工具，因此非常適合以語言為中心、需要彈性的工作。如要進一步瞭解 LLM 代理程式，請參閱[這個頁面](#)。

請修改 `agent.py` 的程式碼，如下所示：

```
from google.adk.agents import Agent

root_agent = Agent(
    model='gemini-2.5-flash',
    name='hotel_agent',
    description='A helpful assistant that answers questions about a specific city.',
    instruction='Answer user questions about a specific city to the best of your knowledge. Do not answer questions outside of this.',
)
```

在本機測試 Agent 應用程式

在現有的終端機視窗中輸入下列指令。確認您位於包含 `hotel_agent_app` 資料夾的上層資料夾 (`my-agents`) 中。

```
adk web
```

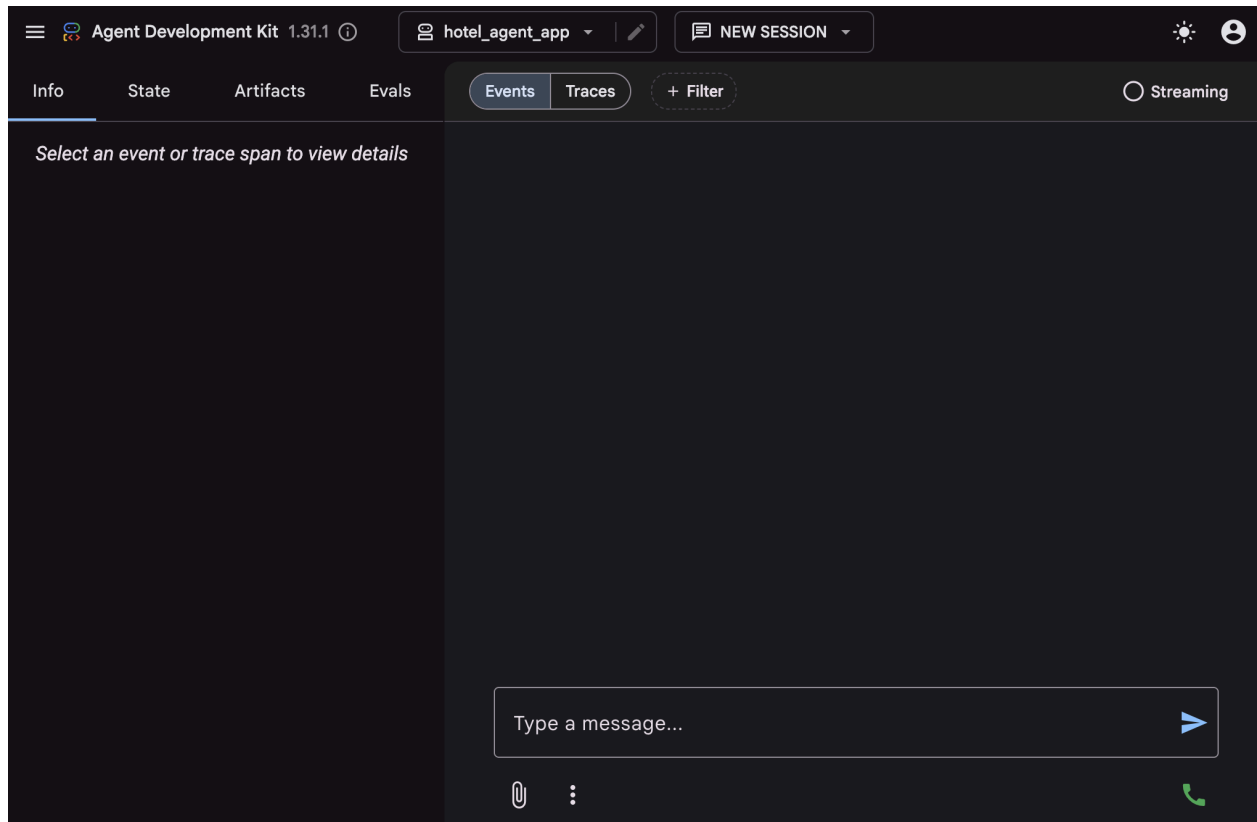
執行範例如下所示：

```
INFO:      Started server process [1478]
INFO:      Waiting for application startup.

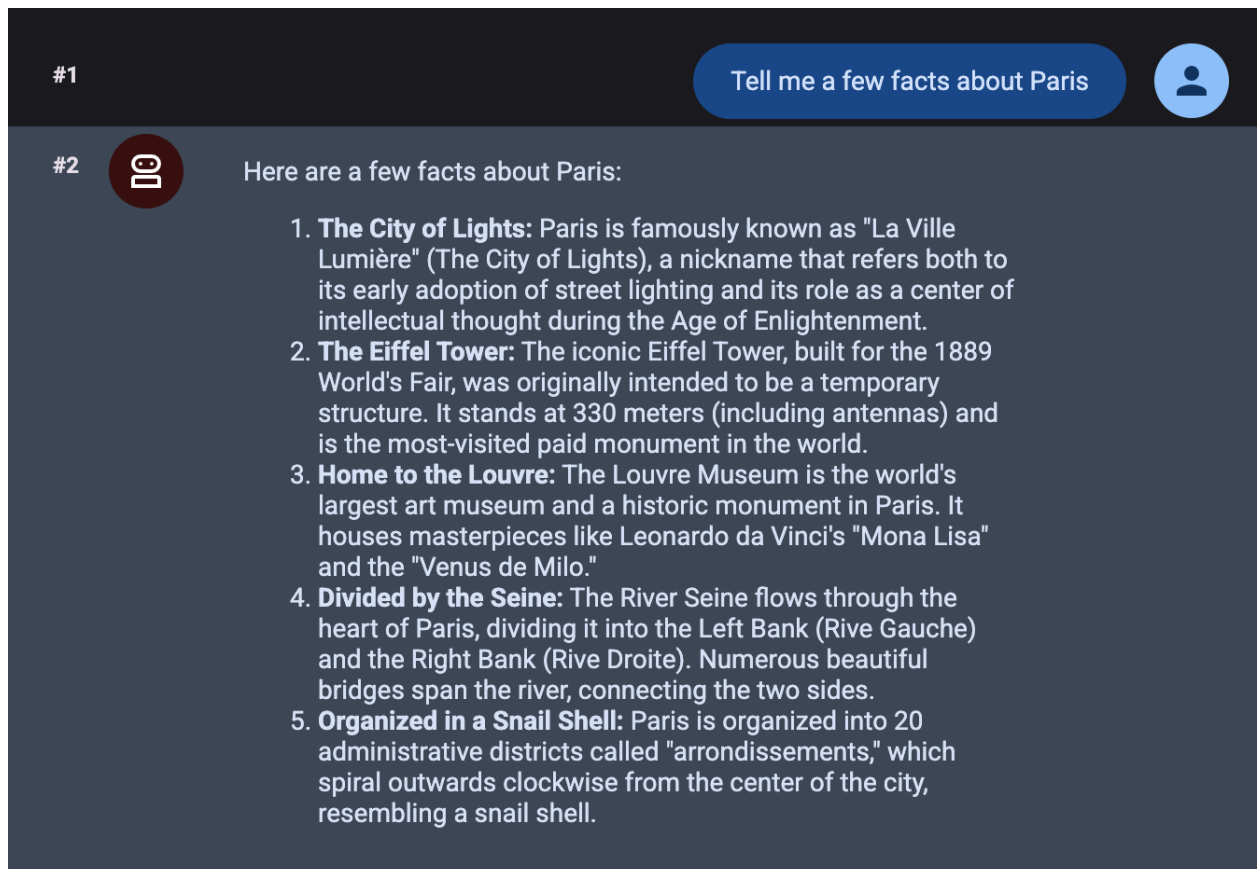
+-----+
| ADK Web Server started                                     |
|                                                         |
| For local testing, access at http://127.0.0.1:8000.      |
+-----+

INFO:      Application startup complete.
INFO:      Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
```

按一下最後一個連結，系統應會顯示網頁控制台，供您測試代理程式。瀏覽器中應會啟動下列項目，如下所示：



請注意，左上方已識別出 `hotel_agent_app`。現在可以開始與代理對話。請提供幾個詢問城市相關資訊的提示。對話範例如下所示：



您可以關閉在 Cloud Shell 終端機中執行的程序 (Ctrl-C)。

如要測試 Agent，也可以使用 `adk run` 指令，如下所示 (位於 `my-agents` 資料夾中)。

```
adk run hotel_agent_app
```

試試看這項指令，即可透過指令列 (終端機) 與代理程式對話。輸入 **exit** 即可關閉對話。

8. 將代理程式連結至工具

我們已瞭解如何編寫代理程式並在本機測試。我們將這個代理程式連結至工具。在 ADK 架構中，工具代表賦予 AI 代理的特定能力，使其能執行各種動作並與世界互動，而不侷限於核心的文字生成和推論能力。

在本例中，我們現在要為代理提供在 MCP Toolbox for Databases 中設定的工具。

使用下列程式碼修改 `agent.py` 檔案。請注意，我們在程式碼中使用預設通訊埠 5000，但如果您使用其他通訊埠號碼，請使用該號碼。

```
from google.adk.agents import Agent
from toolbox_core import ToolboxSyncClient

toolbox = ToolboxSyncClient("http://127.0.0.1:5000")

# Load single tool
# tools = toolbox.load_tool('search-hotels-by-location')

# Load all the tools
tools = toolbox.load_toolset('my_first_toolset')

root_agent = Agent(
    name="hotel_agent",
    model="gemini-2.5-flash",
    description=(
        "Agent to answer questions about hotels in a city or hotels by name."
    ),
    instruction=(
        "You are a helpful agent who can answer user questions about the hotels in a specific city or hotels by name. Use the tools to answer the question"
    ),
    tools=tools,
)
```

現在可以測試代理，從已透過 MCP Toolbox for Databases 設定的 PostgreSQL 資料庫擷取實際資料。

如要執行這項操作，請依序完成下列步驟：

在 Cloud Shell 的其中一個終端機中，啟動 MCP Toolbox for Databases。如先前測試時一樣，您可能已在本機的 5000 埠執行此服務。如果沒有，請執行下列指令 (從 `mcp-toolbox` 資料夾) 啟動伺服器：

```
./toolbox --config "tools.yaml"
```

理想情況下，您應該會看到伺服器已連線至資料來源，並載入工具集和工具的輸出內容。

MCP 伺服器成功啟動後，請在另一個終端機中啟動 Agent，方法與先前相同，透過下列 `adk run` (來自 `my-agents` 資料夾) 指令啟動。您也可以視需要使用 `adk web` 指令。

```
$ adk run hotel_agent_app/

...
Running agent hotel_agent, type exit to exit.
[user]: what can you do for me ?
[hotel_agent]: I can help you find hotels by location or by name.
[user]: I would like to search for hotels?
[hotel_agent]: Great! Do you want to search by location or by hotel name?
[user]: I'd like to search in Basel
[hotel_agent]: Here are some hotels in Basel:

*   Holiday Inn Basel (Upper Midscale)
*   Hyatt Regency Basel (Upper Upscale)
*   Hilton Basel (Luxury)
[user]:
```

請注意，Agent 現在會使用我們在 MCP Toolbox for Databases 中設定的兩個工具 (**search-hotels-by-name** 和 **search-hotels-by-location**)，並提供正確的選項。接著，即可從 PostgreSQL 執行個體資料庫順利擷取資料，並相應地設定回應格式。

這樣就完成了使用 Agent Development Kit (ADK) 建構的飯店代理本機開發和測試，該代理是由我們在 MCP Toolbox for Databases 中設定的工具提供支援。

9. (選用) 將 MCP Toolbox for Databases 和 Agent 部署至 Cloud Run

在上一節中，我們使用 Cloud Shell 終端機啟動 MCP Toolbox 伺服器，並透過 Agent 測試工具。這是在 Cloud Shell 殼層環境中在本機執行的。

您可以選擇將 MCP Toolbox 伺服器和代理程式部署至 Google Cloud 服務，由這些服務代為代管應用程式。

在 Cloud Run 託管 MCP Toolbox 伺服器

首先，我們可以從 MCP Toolbox 伺服器開始，並將其託管於 Cloud Run。這樣一來，我們就能取得公用端點，並與任何其他應用程式和/或代理程式應用程式整合。如需在 Cloud Run 上代管這項服務的操作說明，請參閱[這篇文章](#)。我們現在來瞭解重要步驟。

啟動新的 Cloud Shell 終端機，或使用現有的 Cloud Shell 終端機。前往 `mcp-toolbox` 資料夾，其中包含 `toolbox` 二進位檔和 `tools.yaml`。

執行下列指令 (每個指令都有相關說明)：

將 `PROJECT_ID` 變數設為指向您的 Google Cloud 專案 ID。

```
export PROJECT_ID="YOUR_GOOGLE_CLOUD_PROJECT_ID"
```

接著，請確認專案中已啟用下列 Google Cloud 服務。

```
gcloud services enable run.googleapis.com \
                        cloudbuild.googleapis.com \
                        artifactregistry.googleapis.com \
                        iam.googleapis.com \
                        secretmanager.googleapis.com
```

我們將建立個別的服務帳戶，做為要在 Google Cloud Run 部署的 Toolbox 服務身分。我們也會確保這個服務帳戶具備正確的角色，也就是有權存取 Secret Manager 並與 Cloud SQL 通訊。

```
gcloud iam service-accounts create toolbox-identity

gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member serviceAccount:toolbox-identity@$PROJECT_ID.iam.gserviceaccount.com \
  --role roles/secretmanager.secretAccessor

gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member serviceAccount:toolbox-identity@$PROJECT_ID.iam.gserviceaccount.com \
  --role roles/cloudsql.client
```

我們將會將 `tools.yaml` 檔案上傳為密鑰，由於我們必須在 Cloud Run 中安裝 Toolbox，因此會使用 Toolbox 的最新容器映像檔，並在 `IMAGE` 變數中設定該映像檔。

```
gcloud secrets create tools --data-file=tools.yaml

export IMAGE=us-central1-docker.pkg.dev/database-toolbox/toolbox/toolbox:latest
```

最後一個步驟是使用熟悉的部署指令將應用程式部署至 Cloud Run：

```
gcloud run deploy toolbox \
--image $IMAGE \
--service-account toolbox-identity \
--region us-central1 \
--set-secrets "/app/tools.yaml=tools:latest" \
--args="--config=/app/tools.yaml","--address=0.0.0.0","--port=8080" \
--allow-unauthenticated
```

這會開始將設定的 `tools.yaml` 部署至 Cloud Run。部署成功後，您應該會看到類似以下的訊息：

```
Deploying container to Cloud Run service [toolbox] in project [YOUR_PROJECT_ID] region [us-central1]
OK Deploying new service... Done.

OK Creating Revision...

OK Routing traffic...

OK Setting IAM Policy...

Done.

Service [toolbox] revision [toolbox-00001-zsk] has been deployed and is serving 100 percent
of traffic.
Service URL: https://toolbox-<SOME_ID>.us-central1.run.app
```

現在可以在瀏覽器中前往上述列出的 `Service URL`。應該會顯示先前看到的「Hello World」訊息。

您也可以從 Google Cloud 控制台前往 [Cloud Run](#)，即可在 Cloud Run 的服務清單中看到 Toolbox 服務。

注意：如要繼續在本機執行飯店代理程式，但連線至新部署的 Cloud Run 服務，您只需要在 `my-agents/hotel_agent_app/agent.py` 檔案中進行一項變更。

請勿使用以下程式碼：

```
toolbox = ToolboxSyncClient("http://127.0.0.1:5000")
```

將其變更為 Cloud Run 服務的服務網址，如下所示：

```
toolbox = ToolboxSyncClient("CLOUD_RUN_SERVICE_URL")
```

使用 `adk run` 或 `adk web` 測試 Agent 應用程式，如先前所示。

在 Cloud Run 上部署飯店代理程式應用程式

首先，請按照上述指示，確保您已在 `my-agents/hotel_agent_app/agent.py` 中進行變更，指向在 Cloud Run 上執行的 Toolbox 服務網址，而非本機主機。

在新的 Cloud Shell 終端機或現有的終端機工作階段中，確認您位於先前設定的正確 Python 虛擬環境。

首先，請在 `my-agents/hotel_agent_app` 資料夾中建立 `requirements.txt` 檔案，如下所示：

```
google-adk
toolbox-core
```

前往 `my-agents` 資料夾，並先設定下列環境變數：

```
export GOOGLE_CLOUD_PROJECT=YOUR_GOOGLE_CLOUD_PROJECT_ID
export GOOGLE_CLOUD_LOCATION=us-central1
export AGENT_PATH="hotel_agent_app/"
export SERVICE_NAME="hotels-service"
export APP_NAME="hotels-app"
export GOOGLE_GENAI_USE_VERTEXAI=True
```

最後，請使用下列 `adk deploy cloud_run` 指令，將代理程式應用程式部署至 Cloud Run。如果系統要求允許未經驗證就叫用服務，請暫時提供「y」做為值。

```
adk deploy cloud_run \
--project=$GOOGLE_CLOUD_PROJECT \
--region=$GOOGLE_CLOUD_LOCATION \
--service_name=$SERVICE_NAME \
--app_name=$APP_NAME \
--with_ui \
$AGENT_PATH
```

系統會開始將飯店代理程式應用程式部署至 Cloud Run。這項工具會上傳來源、封裝至 Docker 容器、將容器推送至 Artifact Registry，然後在 Cloud Run 上部署服務。這可能需要幾分鐘的時間，請耐心等待。

畫面會顯示類似以下的訊息：

```
Start generating Cloud Run source files in /tmp/cloud_run_deploy_src/20250905_132636
Copying agent source code...
Copying agent source code completed.
Creating Dockerfile...
Creating Dockerfile complete: /tmp/cloud_run_deploy_src/20250905_132636/Dockerfile
Deploying to Cloud Run...
Building using Dockerfile and deploying container to Cloud Run service [hotels-service] in
project [YOUR_PROJECT_ID] region [us-central1]
- Building and deploying... Uploading sources.

- Uploading sources...

. Building Container...

OK Building and deploying... Done.

OK Uploading sources...

OK Building Container... Logs are available at [https://console.cloud.google.com/cloud-
build/builds;region=us-central1/d1f7e76b-0587-4bb6-b9c0-bb4360c07aa0?project=415
458962931].

f

OK Creating Revision...

OK Routing traffic...

Done.

Service [hotels-service] revision [hotels-service-00003-hrl] has been deployed and is serving
100 percent of traffic.
Service URL: <YOUR_CLOUDRUN_APP_URL>
INFO: Display format: "none"
Cleaning up the temp folder: /tmp/cloud_run_deploy_src/20250905_132636
```

部署成功後，系統會提供服務網址的值，您可以在瀏覽器中存取該網址，查看與飯店服務專員對話的相同 Web 應用程式，如先前在本地設定中所示。

☰

hotels-app

EventsStateArtifactsSessionsEval

Conversations with agent

0 model:0Hello! How can I help you with hotels today?

1 model:functionCall:1search-hotels-by-location

2 user:functionResponse:2search-hotels-by-location

3 model:3OK. I found three hotels in Basel: Hilton Basel, Hya...

SESSION ID 03c8afa5-a47c-4a5c-a361-985beef1b966

Token Streaming | + New Session

hello

can you help me with hotels in Basel

Hello! How can I help you with hotels today?

search-hotels-by-location

search-hotels-by-location

OK. I found three hotels in Basel: Hilton Basel, Hyatt Regency Basel, and Holiday Inn Basel.

Type a Message...

10. 清除所用資源

為避免系統持續向您的 Google Cloud 帳戶收費，請務必刪除我們在研討會期間建立的資源。我們將刪除 Cloud SQL 執行個體，如果您已將工具箱和飯店應用程式部署至 Cloud Run，我們也會刪除這些服務。

請根據專案和區域，確認下列環境變數設定正確：

```
export PROJECT_ID="YOUR_PROJECT_ID"
export REGION="YOUR_REGION"
```

下列兩個指令會刪除我們部署的 Cloud Run 服務：

```
gcloud run services delete toolbox --platform=managed --region=${REGION} --
project=${PROJECT_ID} --quiet

gcloud run services delete hotels-service --platform=managed --region=${REGION} --
project=${PROJECT_ID} --quiet
```

下列指令會刪除 Cloud SQL 執行個體：

```
gcloud sql instances delete hoteldb-instance
```

11. 恭喜

恭喜！您已成功使用 Agent Development Kit (ADK) 建構旅遊代理，並運用 MCP Toolbox for Databases。您也學會如何將 Agent 連線至自己的資料庫，以及如何在 Cloud Run 上安裝 Agent 和工具箱 (選用)。

參考文件

- [Agent Development Kit](#)
 - [MCP Toolbox for Databases](#)
 - [從 IDE 連線至 MCP Toolbox](#)
 - [PostgreSQL 適用的 Cloud SQL](#)
-